

Desenvolvimento para Dispositivos Móveis

Trabalho de Implementação

Universidade Federal do Rio Grande do Norte

Descrição

- Trabalho individual
- Nota máxima 6,0
- Aula 24/09/2026 dúvidas
- Entrega 29/09/2026
 - Apresentação individual no horário de aula.
 - Usar emulador ou dispositivo físico.
 - Explicação do código-fonte.

Detalhes

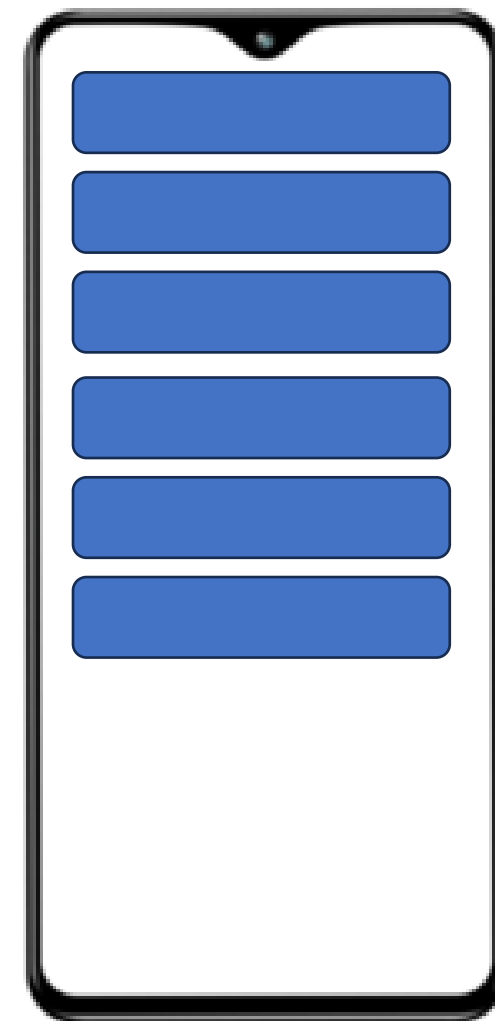
- Implementar um aplicativo para consumir uma API gerada com os dados do seguinte repositório:
 - <https://cdn.jsdelivr.net/gh/akabab/superhero-api@0.3.0/api/all.json>
 - Você deve baixar o arquivo JSON e executar um servidor local com seus dados simulando uma API REST.
 - Utiliza a biblioteca <https://www.npmjs.com/package/json-server>
- O aplicativo deve ter design harmonioso e responsivo.

- Tela inicial
 - Agentes: Abre o catálogo geral com a listagem de todos os heróis disponíveis na API
 - Contrato Diário: Abre a tela de convocação diária para recrutar novos heróis.
 - Meu Esquadrão: Abre a listagem dos agentes recrutados na equipe.
 - Missões: Abre a central tática de simulação de missões e combate.



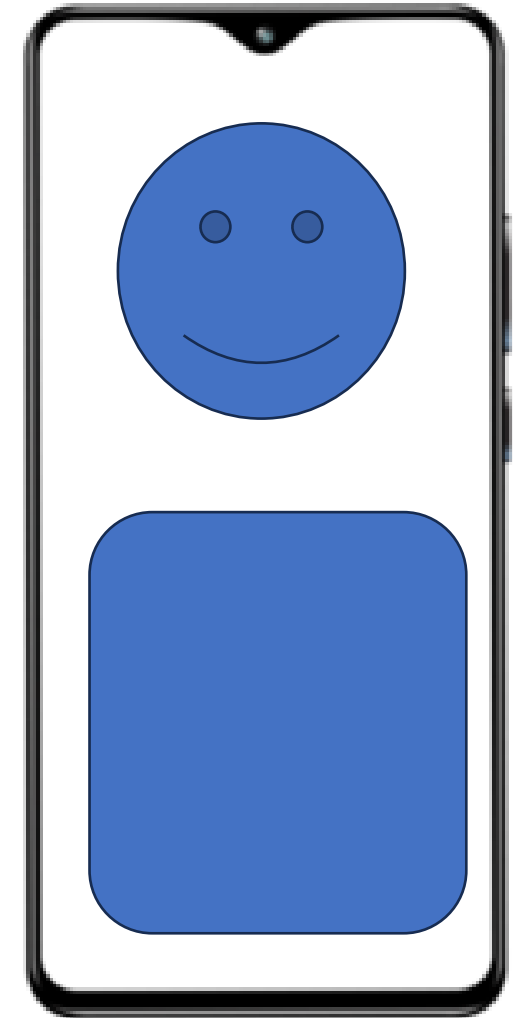
Tela Agentes (Catálogo Geral)

- Lista todos os heróis em uma PagedListView utilizando a biblioteca infinite_scroll_pagination
- Ao invocar a API o aplicativo deve salvar em cache todas as informações no banco de dados. (ex.: SQLite/sqlflite).
- Se o aplicativo estiver sem internet, o aplicativo deve exibir informações com base no cache do banco de dados.
- Os cards devem conter informações como nome, powerstarts e appearance dos heróis (não precisa ser tudo).
- Exiba a imagem miniatura de cada herói no card (cached_network_image).
- O clique no card abre a tela de **Detalhes do Agente**.



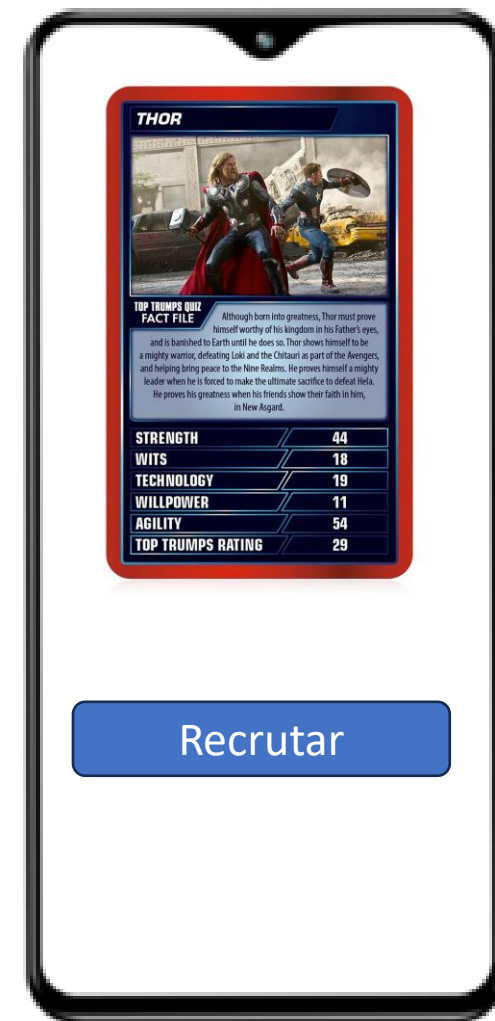
Tela Detalhes Agente

- Exibe a imagem do herói em alta resolução.
- Exibe todos os detalhes do herói que questão disponíveis na API
- Os dados devem ser lidos preferencialmente da API ou recuperados do banco local em caso offline.
- Utilize a biblioteca `cached_network_image` para fazer cache das imagens.
- Utilize a biblioteca [`primer\_progress\_bar`](#) para exibir uma progress bar de cada powerstat (intelligence, strength, speed, durability, power, combat) .



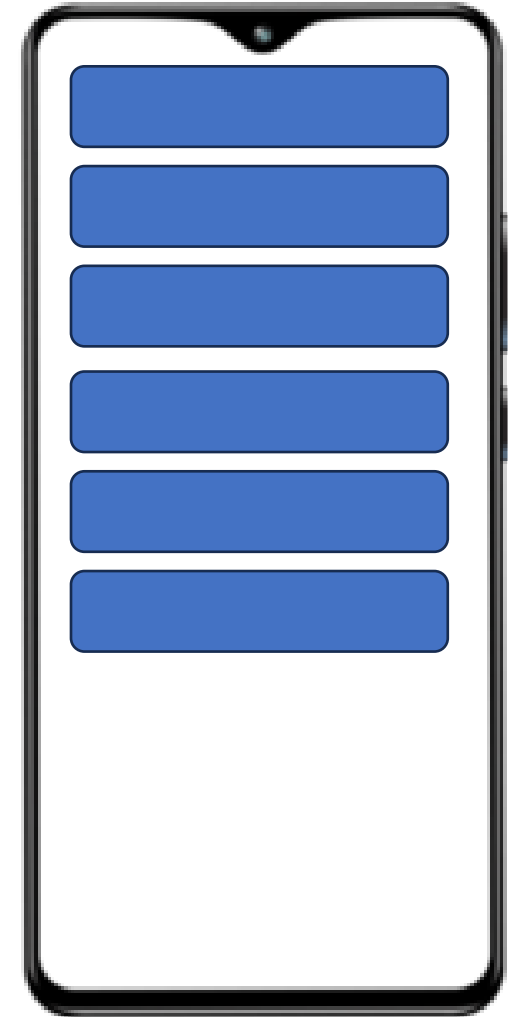
Tela Contrato Diário (Recrutamento)

- Uma vez ao dia, o app apresenta um card de herói aleatório disponível para contratação no esquadrão. Deve conter apenas nome, imagem e power stats.
- Utilizar `shared_preferences` (ou banco de dados) para armazenar a data do último sorteio e o ID do herói convocado no dia.
- Botão "**Recrutar para o Esquadrão**":
 - Adiciona o herói à tabela de membros do esquadrão no banco local.
 - O esquadrão suporta no máximo **15 agentes**.
 - Caso o usuário já possua 15 agentes, o botão deve ser desabilitado ou disparar mensagem de erro informando que o esquadrão atingiu a capacidade máxima.



Tela Meu Esquadrão

- Lista exclusivamente os heróis (até 15) que já foram recrutados e salvos no banco de dados local.
- Exibe cards com nome, papel tático ou maior atributo, e miniatura com `cached_network_image`
- Um toque no card leva o usuário para a tela **Detalhes do Meu Agente**.



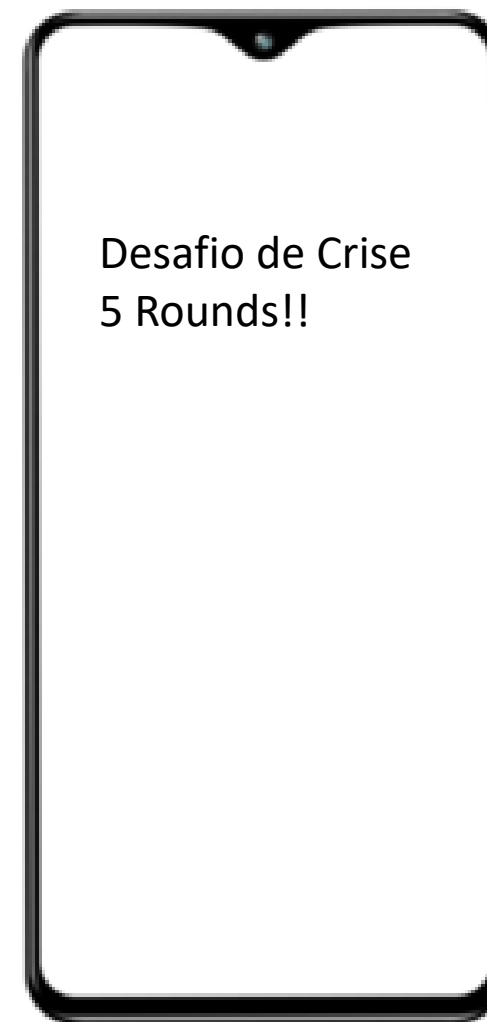
Tela Detalhes do Meu Agente

- Exibe a carta o herói em todos os detalhes.
- As informações devem ir da API ou do cache local.
- Botão “Dispensar do Esquadrão” remove o agente do banco de dados local, liberando vaga no esquadrão.
 - Exiba uma caixa de diálogo para que o usuário confirme a ação. Use a biblioteca `awesome_dialog`.
- Utilize a biblioteca `cached_network_image` para fazer cache das imagens.



Tela Iniciar Missão

- A tela de Missão permite ao jogador testar o nível tático do seu esquadrão contra ameaças do sistema.
- O esquadrão precisa ter pelo menos 5 agentes para iniciar uma Missão.
- O app sorteia aleatoriamente um **Desafio de Crise** contendo 3 a 5 rounds.
- Em cada round é exigido um atributo dominante de teste (ex.: *Intelligence*, *Strength*, *Speed* ou *Combat*).
- O app sorteia um oponente do catálogo geral da API/banco para atuar como o "Inimigo da Rodada".
 - Se sortear um herói do esquadrão do usuário, deve-se sortear novamente.



Como Batalhar

- **Apresentação do Desafio**

- O app exibe a imagem e o nome do inimigo (não exibe seus atributos) além do nome do atributo em disputa que foi escolhido.

- **Escalação**

- O jogador seleciona 1 dos seus agentes disponíveis no esquadrão para enfrentar aquela etapa específica.
- Miniaturas apenas com nome e imagem circular em um grid 3x5

- **Resolução:**

- Ao confirmar o agente, o app compara o valor do atributo exigido entre o Herói e o Inimigo:
- Se Atributo do Herói > Inimigo = Sucesso na Rodada.
- Se Atributo do Herói < Inimigo = Falha na Rodada.
- Se Valores Iguais = Empate tático.

Execução e Controle dos Rounds

- Após a rodada o jogador visualiza os indicadores de resultado calculados pelo app (**Vitória**, **Derrota** ou **Empate**).
- Cada agente do esquadrão só pode ser enviado para **uma rodada por missão**.
- Após o confronto da rodada, o app atualiza o placar e avança automaticamente para o próximo round da fila até concluir a campanha.

Fim da Missão e Feedback

- A missão termina quando todos os rounds da operação forem finalizados.
- O app exibe o sumário com:
 - Total de vitórias obtidas pelo esquadrão.
 - Total de derrotas/falhas sofridas.
- O encerramento deve disparar uma caixa de diálogo personalizada via `awesome_dialog`:
 - **Tipo Sucesso (`DialogType.success`):** Se venceu mais da metade dos rounds, informando "Missão Cumprida!". É sorteado um dos heróis que participou da vitória que ganha +1 em um powerstat aleatório. Uma imagem do herói é exibida.
 - **Tipo Erro/Aviso (`DialogType.error` ou `warning`):** Se perdeu a maioria, informando "Operação Fracassada!". Uma imagem de derrota é exibida.

Resumo dos Critérios de Avaliação

- **Arquitetura e Offline-First:** Persistência em cache local e funcionamento completo sem internet (após primeira carga da API).
- **Bibliotecas Obrigatórias Integradas:**
 - json-server (backend mock)
 - infinite_scroll_pagination (listagem infinita)
 - cached_network_image (cache de imagens)
 - primer_progress_bar (barras de atributos)
 - awesome_dialog (confirmações e feedback da missão)
 - shared_preferences e SQLite (controle diário e esquadrão)
- **Regras de Negócio:** Limite de 15 agentes, sorteio 1x por dia, bloqueio de agente duplicado na mesma missão.